

City University of Hong Kong 2025-2026
Semester A CS3343 Software Engineering
Practice

Bug Report
Group 19
Classic UNO Card Game

Conducted By:

Woo King Hei 58533231

Wong Ka Chung 58533673

Chan Pak Yuen 58535918

Chan Wing Ki 58111083

Ho Sung Hin 58010837

Wong Hei Leong 58537010



1. Introduction.....	3
1.1 Repository.....	4
2. Guidelines.....	4
2.1 Description.....	4
2.2 Steps to Reproduce.....	4
2.3 Expected Result.....	4
2.4 Actual Result.....	4
2.5 Related Code.....	4
2.6 Status and Result.....	5
2.7 Severity.....	5
2.8 Priority.....	6
2.9 Hardware and Operating System.....	6
2.10 Reporter and Assignee.....	6
2.11 Comments.....	6
3. Reported Bugs.....	7
3.1 The getTopCard() function Will End Up Checking the Wild Draw 4 Card Instead.....	7
Description.....	7
Steps to Reproduce.....	7
Related Code.....	7
Expected Result.....	8
Actual Result.....	8
3.2 CPUPlayers Can Draw Cards Outside of Their Turn.....	10
Description.....	10
Steps to Reproduce.....	10
Related Code.....	11
Expected Result.....	11
Actual Result.....	11
3.3 Feature Request: Sorting Algorithm for Cards in Hand.....	13
Description.....	13
Expected Result.....	13
3.4 Card Playing Logic Error After Wild Card is Played.....	15
Description.....	15
Steps to Reproduce.....	15
Related Code.....	16
Expected Result.....	16
Actual Result.....	17
3.5 WildCard Timeout Operation.....	19
Description.....	19
Related Code.....	20
Expected Result.....	20

Actual Result.....	20
3.6 Cannot shout UNO when other player's turn.....	23
Description.....	23
Related Code.....	24
Expected Result.....	24
Actual Result.....	24
3.7 "Null Card" is Being Added to the Hand When the Deck is Empty.....	27
Description.....	27
Steps to Reproduce.....	27
Related Code.....	28
Expected Result.....	28
Actual Result.....	28
3.8 The Top Card is Not a Number Card During Game Initialization.....	30
Description.....	30
Steps to Reproduce.....	30
Related Code.....	30
Expected Result.....	30
Actual Result.....	30
3.9 Game Does Not End When Current Player has No Cards in Hand.....	32
Description.....	32
Steps to Reproduce.....	32
Related Code.....	32
Expected Result.....	33
Actual Result.....	33
3.10 HumanPlayer Can Draw Cards Multiple Times During Their Turn.....	35
Description.....	35
Steps to Reproduce.....	35
Related Code.....	36
Expected Result.....	36
Actual Result.....	37
4. Appendix.....	39
4.1 Bug Report Template.....	39

1. Introduction

During the development process for the UNO game, we encountered a number of bugs that needed to be fixed. This document is a bug report which stores information on the bugs we faced while creating the UNO game and documents the fixing process of each recorded bug.

For each bug, a description is provided to provide context on what effect does the bug have on the game. Steps to reproduce the bug are also written to better test out the bug and give the assignee a better understanding on how the bug occurs. For additional context of the bug, expected and actual results are also outlined. If possible for the reporter, related code will also be pointed out to aid the assignee in identifying the root cause faster.

1.1 Repository

For more efficient reporting and communication, our group used BugZilla as our bug repository. BugZilla is an open source system which supports time-tracking, communication and formatted bug reports. With the help of BugZilla, our bug reports are organized with a fixed template, improving readability. Communication between teammates regarding each bug also became more efficient, as each bug has its own independent channel.

The screenshot shows a Bugzilla bug report for Bug 932, titled "Cannot shout UNO during other player's turn". The status is "VERIFIED FIXED". The reporter is wkchan597-c@my.cityu.edu.hk, and it was modified on 2025-11-26 23:08 HKT. The bug is assigned to Ken Wong. The product is "25/26 CS3343 Group 19 UNO Game", the component is "HumanPlayer", and the version is "v1.0". The hardware is "Macintosh" and the operating system is "Mac OS". The importance is "major". The bug has no URL, personal tags, depends on, or blocks. The "CC List" is empty, and the "Ignore Bug Mail" option is selected.

Figure

The screenshot shows a Bugzilla bug report form for a new bug. The form includes fields for Product, Component, Version, Severity, Hardware, OS, and Summary. The Product is "25/26 CS3343 Group 19 UNO Game", the Component is "CPUPlayer", the Version is "v1.0", the Severity is "critical", the Hardware is "PC", and the OS is "Windows". The Summary is "CPUPlayer executes HumanPlayer Method". The form also includes a "Component Description" field and a "Summary" field. The form is titled "Before reporting a bug, please read the bug writing guidelines, please look at the list of most frequently reported bugs, and please search for the bug." and includes a "Show Advanced Fields" link.

2. Guidelines

To better navigate this bug report, this section will introduce some guidelines that will be used in this bug report for clearer communication and organization.

2.1 Description

The description section provides an explanation and context for the bug. The reporter may include what caused the bug to occur and how the bug affects the operation of the UNO game. This part serves a key role in communicating the impact that the bug has on the UNO game.

2.2 Steps to Reproduce

The reporter will also outline the steps to reproduce the bug for more efficient testing of the bug. Screenshots (if available) should also be provided to better explain the situation for the assignee to understand. This part is important for the assignee to know how to repeat the process, which leads to finding the root cause of the bug.

2.3 Expected Result

The reporter is responsible for illustrating the preferred outcome when using the relevant component/function. For feature requests, this part will explain the ideal results after implementing the enhancement feature.

2.4 Actual Result

This part is for the reporter to explain the actual outcome when the bug occurs and affects the relevant component/function.

2.5 Related Code

If the reporter can find the code which caused the error, mentioning it in the bug report helps the reporter to fix the issue faster. The reporter should only paste the code of the related component (s), but not copy and paste the code of an entire class.

2.6 Status and Result

The status of each bug always starts with “UNCONFIRMED”. As the bug gets confirmed and the assignee starts working on it, the status may change to “OPENED”, “IN PROGRESS” or “CLOSED” depending on the situation. If the bug ever reoccurs after being fixed, the status will be changed to “REOPENED”.

UNCONFIRMED	No one has discovered the bug yet.
OPENED	A tester has discovered and reported the bug.
IN PROGRESS	The assignee is working on the bug.
CLOSED	The bug is fixed and the problem is confirmed to be solved via re-testing.

REOPENED	The bug recurred after the bug was fixed.
----------	---

The result status represents the final result of fixing the bug. Depending on the fixing outcome, the resolution of the bug can be “FIXED”, “NOT FIXED”, “WORKS FOR ME”. For special cases, “NOT A BUG” and “REPEATED BUG” will be used depending on the situation.

FIXED	The bug fix was successful.
NOT FIXED	The bug fix was unsuccessful and will have to resolve it in future updates.
WORKS FOR ME	No one besides the reporter can reproduce the bug.
NOT A BUG	The reported issue is not a bug. It could be an issue on the reporter’s end instead of the code.
REPEATED BUG	The issue mentioned is already covered in another reported bug.

2.7 Severity

The severity of the bug represents the impact level that the bug has on the UNO game.

CRITICAL	The bug greatly disrupts the development of the UNO game.
SIGNIFICANT	The bug largely impacts the operation of the UNO game.
MODERATE	The bug affects some functionality of a component
MINOR	The bug only causes minor inconvenience in the operation of the UNO game.
ENHANCEMENT	The game works fine, it is a feature request to enhance the game operation.

2.8 Priority

The priority of the bug shows the level of urgency for fixing the bug. Typically speaking, the bugs with higher severity levels will have a high priority as well.

!IMPORTANT	The bug has the highest priority and needs to be fixed ASAP. It could be a bug that potentially endangers our codebase.
HIGH	The bug disrupts the game operation immensely and needs to be fixed.
MEDIUM	The bug needs to be fixed but not urgent like the above two levels.
LOW	The bug does not affect the game operation in any significant way and can be fixed once the bugs with higher priority are all fixed.

2.9 Hardware and Operating System

The code may perform differently due to different hardware and operating systems. Therefore, when reporting a bug, it is important to point out what OS the reporter is using when encountering the bug. Some bugs may even be unfixable on certain setups.

2.10 Reporter and Assignee

The reporter is the tester who discovered the bug and reported it. Whereas the assignee is the person who is assigned to follow up the reported bug, which oftentimes is the developer who developed the related components. The assignee typically is responsible for fixing the bug for the reporter to re-test the bug. Once the re-test has no problems, the bug is officially resolved.

2.11 Comments

Comments are the communication records between the reporter and the assignee working on the bug. The assignee can ask for more information on the reported bug and the reporter can communicate to the assignee on whether they have successfully fixed the issue, then update the status and result of the bug.

3. Reported Bugs

3.1 The getTopCard() function Will End Up Checking the Wild Draw 4 Card Instead

Product	UNO
Status	CLOSED FIXED
Component	Card.getTopCard()
Version	1.0
Hardware	Macintosh
OS	Mac OS
Severity	Significant
Priority	High
Reporter	Woo King Hei <kingwoo8-c@my.cityu.edu.hk>
Assignee	Wong Hei Leong <hlwong397-c@my.cityu.edu.hk>

Woo King Hei 2025-10-25 23:12:27 HKT

Description

When testing the program, I find the coding logic for the challenge function being flawed. Whenever I (HumanPlayer) or the CPU Player (CPUPlayer) challenges a Wild Draw Four, the challenge always succeeds even if I had no playable cards other than the Wild Draw Four Card.

Steps to Reproduce

1. Play a Wild Draw Four Card legally when you have ≥ 5 cards left in hand.
2. You will get challenged by the next CPU Player, which will succeed instead of failing according to game rules.

Please look into this issue and fix it as soon as possible.

Related Code

```
public Card getTopCard() {  
    return PlayedCard.get(PlayedCard.size() - 1);  
}
```

Expected Result

In this game, I had no other blue cards and must play the Wild Draw Four Card in my hand.



CPU3 decides to challenge since I had equal or more than 5 cards left in my hand. The challenge should fail since I don't have any blue cards.



Actual Result

However, the challenge succeeded, and I had to draw 4 cards.



Here is the game log for reference:

```
UNO Game window created and should be visible!
Current Player: Player
Player plays Blue 3
CPU1 (CPU) plays Blue 9
CPU2 (CPU) plays Blue 9
CPU3 (CPU) plays Blue 5
Player plays Wild Draw Four
Wild card played: change color
Change to Red
CPU1 (CPU) challenges Player's Wild Draw Four card!
Challenge Success!
CPU1 (CPU) has no valid cards to play and must draw.
CPU2 (CPU) plays Red Draw Two
Draw Two card played: next player draws 2 cards
```

Wong Hei Leong 2025-10-25 23:30:35 HKT

Received your bug report.

Wong Hei Leong 2025-10-26 10:12:14 HKT

I have now found the issue, the issue lies within the `getTopCard()` function, the function will check the current top card, which is the Wild Draw Four Card that was just played. As a result, the challenge will always succeed. I will handle this issue and update you once fixed.

Wong Hei Leong 2025-10-26 12:25:43 HKT

I have fixed the issue. The challenge function should now work as expected. I have modified the `getTopCard()` function so it checks the current (index)th top card (for example if `index == 2`, it will check the second card right below the Wild Draw Four Card). Now it should work normally.

```
public Card getTopCard(int index) {
    if (index <= 0 || index > PlayedCard.size()) {
        System.out.println("Warning: Invalid card index " +
            index + ", PlayedCard size is " + PlayedCard.size());
        return null;
    }
    return PlayedCard.get(PlayedCard.size() - index);
}
```

Woo King Hei 2025-10-26 13:03:26 HKT

The challenge now works as expected, thanks for your help!

3.2 CPU Players Can Draw Cards Outside of Their Turn

Product	UNO
Status	CLOSED FIXED
Component	UNOController.java
Version	1.0
Hardware	Macintosh
OS	Mac OS
Severity	Moderate
Priority	Medium
Reporter	Woo King Hei <kingwoo8-c@my.cityu.edu.hk>
Assignee	Wong Hei Leong <hlwong397-c@my.cityu.edu.hk>

Woo King Hei 2025-10-26 16:34:57 HKT

Description

When testing the program, I played a card on my turn as normal. After that, I tried to click the deck on the left to see if I could draw cards outside of my turn. Turns out that by clicking the deck outside of my turn, CPU Players will draw cards as many times as I click the deck.

Steps to Reproduce

1. Play a card legally then click the deck on the top-left corner outside of your turn.
2. CPU Players will draw cards equal to the number of times you clicked the deck.

Please look into this issue and fix it as soon as possible.

Related Code

```
public void getCardFromDeck() {
    if (currentPlayer != null) {
        System.out.println(currentPlayer.getName() + " draws a card from the deck.");
        currentPlayer.drawCard(
            cardFactory.giveCard(Deck, isTop: false,
                checkCurrentPlayer() == 0 ? true : false, cardString: ""));
        Card card = currentPlayer.getHand().get(currentPlayer.getHand().size() - 1);
        gamePanel.updateDisplay();
        if (canPlayCard(card, getTopCard())) {
            if (checkCurrentPlayer() == 0) {
                deckPlayCardViewer.setIsDeciding(card);
                turnTimer.startTimer(seconds: 10);
            } else {
                System.out.println(currentPlayer.getName() +
                    " got a matching card! they choose to play the card");
                currentPlayer.playCard(card);
            }
        } else {
            passNextPlayer(amount: 1);
            eachRound();
        }
    } else {
        System.err.println(x: "currentPlayer variable is null in getCardFromDeck()");
    }
}
```

Expected Result

In this game, I played the Red Skip card and passed the turn. So CPU3 should be skipped. I then clicked the deck twice afterwards, it should have no effect whatsoever since it's not my turn.



Actual Result

However, the two times I clicked the deck resulted in CPU2 and CPU3 each drawing a card. Since the draw-card action also checks for valid plays, CPU3 was able to play a Wild Card even while skipped.

Here is the game log for better understanding:

```
UNO Game window created and should be visible!
Current Player: Player
Player plays Red Skip
Skip card effect
CPU2 draws a card from the deck.
CPU3 draws a card from the deck.
CPU3 got a matching card! they choose to play the card
CPU3 (CPU) plays Wild Card
CPU2 (CPU) plays Red 1
CPU3 (CPU) plays Red Reverse
ReverseCard function called
CPU3 (CPU) plays Green Reverse
ReverseCard function called
```

Wong Hei Leong 2025-10-26 16:57:27 HKT

Received your bug report.

Wong Hei Leong 2025-10-26 17:28:21 HKT

After reviewing UNOController.java, I have found that the bug is due to not properly implementing turn player checking in the getCardFromDeck() function. To fix this bug, I will add a turn player checking statement in the beginning of the function. I think this can eliminate players from clicking the deck and drawing cards outside of their turn. What do you think? Can you think of a better way?

Woo King Hei 2024-10-26 17:34:45 HKT

Sounds good to me!

Wong Hei Leong 2025-10-26 18:01:43 HKT

I have fixed the issue. The draw card function should now work as expected. It now implements a turn player check so players cannot click the deck and draw cards outside of their turn.

Woo King Hei 2025-10-26 18:17:56 HKT

Yes, now it works properly, thanks!

3.3 Feature Request: Sorting Algorithm for Cards in Hand

Product	UNO
Status	CLOSED FIXED
Version	1.0
Hardware	Macintosh
Component	HumanPlayer.java
Severity	Enhancement
Priority	Low
OS	Mac OS
Reporter	Woo King Hei <kingwoo8-c@my.cityu.edu.hk>
Assignee	Chan Wing Ki <wkchan597-c@my.cityu.edu.hk>

Woo King Hei 2025-10-26 22:45:13 HKT

Description

When playing the game, I find it very time consuming and troublesome to locate the exact card I want to play. Especially when I get hit with a Draw 4 and have many cards in hand. Coupled with the fact that each turn has a countdown timer, it is very easy to misplay when you have a lot of cards in hand. I think a sorting algorithm should be implemented to organize the player's hand each turn.

Expected Result

A sorting algorithm should be implemented so that the player's hand is sorted such that each colour is separated and for each colour, the cards are sorted in ascending order with from number cards to special cards.

Chan Wing Ki 2025-10-26 23:30:35 HKT

Received your feature request.

Chan Wing Ki 2025-10-27 22:37:41 HKT

I have implemented a merge sort algorithm for the player's hand. The sorting algorithm will sort cards in yellow->blue->green->red and ascending order. Hope this helps with the playing experience!

```
private ArrayList<Player> mergeSort(ArrayList<Player> list) {  
    if (list.size() <= 1) {  
        return list;  
    }  
  
    int mid = list.size() / 2;  
    ArrayList<Player> left = new ArrayList<>(list.subList(fromIndex: 0, mid));  
    ArrayList<Player> right = new ArrayList<>(list.subList(mid, list.size()));  
  
    left = mergeSort(left);  
    right = mergeSort(right);  
  
    return merge(left, right);  
}
```

Woo King Hei 2025-10-27 23:05:12 HKT

Yep much better now, thanks!

3.4 Card Playing Logic Error After Wild Card is Played

Product	UNO
Status	CLOSED FIXED
Component	WildCard.checkCard()
Version	1.0
Hardware	Macintosh
OS	Mac OS
Severity	Moderate
Priority	Medium
Reporter	Woo King Hei <kingwoo8-c@my.cityu.edu.hk>
Assignee	Wong Hei Leong <hlwong397-c@my.cityu.edu.hk>

Woo King Hei 2025-10-29 13:27:12 HKT

Description

When the previous player plays a wild card. The next player should only be able to play cards of the declared colour or another wild card. However, while I was testing out the game, I found that CPU Players were able to play any card after a wild card was played. Since it is likely a logic error on WildCard.java itself, I am also pretty certain that Human Players can do that as well.

Steps to Reproduce

1. Play a wild card and declare a colour.
2. CPU Players will play a card with colour different from what you declared with the wild card.

Please look into this issue and fix it as soon as possible.

Related Code

```
@Override
public void cardFunction(UNOController controller) {
    System.out.println(x: "Wild card played: change color");

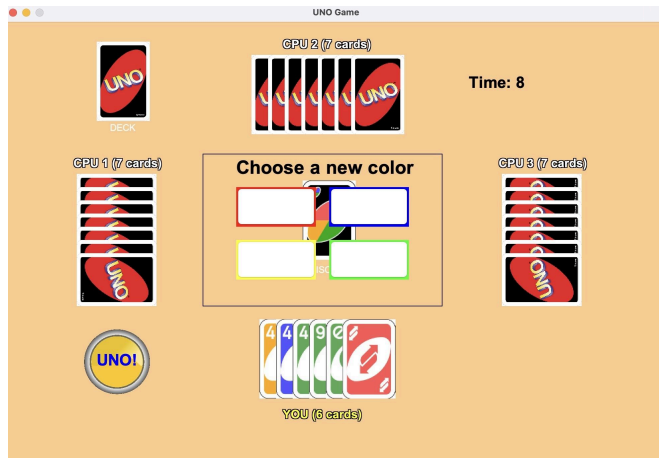
    if (controller.checkCurrentPlayer() == 0) {
        controller.getWildCardViewer().setWildCard(this);
        controller.getTurnTimer().stopTimer();
        controller.getTurnTimer().startTimer(seconds: 10);
    } else {
        Color chooseColor = ((CPUPlayer) controller.getCurrentPlayer()).chooseColor();
        this.setColor(chooseColor);
        this.loadImage("/asset/uno-card-images-master/Wild_Card_Change_Colour_" +
            chooseColor.toString() + ".jpg");
        System.out.println(chooseColor.toString());
        controller.passNextPlayer(amount: 1);
        controller.eachRound();
    }
}

@Override
public boolean checkCard(Card matchCard) {
    /*previous player set the color of wild, so the wild actually has color */
    return true;
}
```

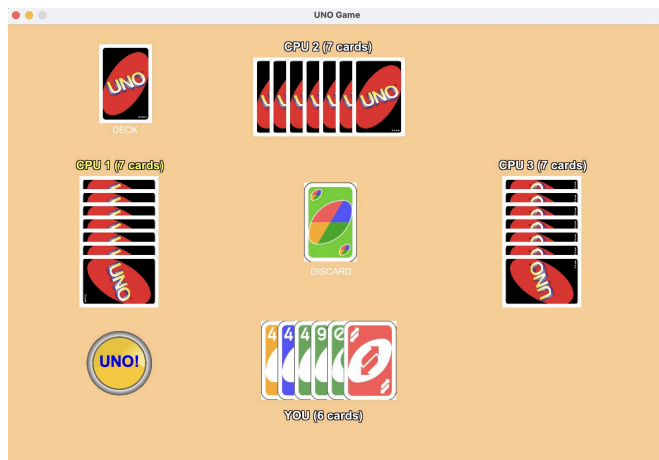
Expected Result

In this game, I played a Wild Card and declared Green





After playing this Wild Card, the next player should only be able to play either green cards or wild cards.



Actual Result

However, the next CPU played a Red card, and my Wild Card became useless.



Here is the game log for better understanding:

```
UNO Game window created and should be visible!  
Current Player: Player  
Player plays Wild Card  
Wild card played: change color  
Change to Green  
CPU1 (CPU) plays Red 0  
CPU2 (CPU) plays Red 3  
CPU3 (CPU) plays Red 7
```

Wong Hei Leong 2025-10-29 13:41:18 HKT

Received your bug report.

Wong Hei Leong 2025-10-29 13:50:22 HKT

I skimmed through the WildCard.java code for a bit. I think the type check function checkCard() looks off, all it does is return true. Maybe that is the reason why the bug occurs. What do you think?

Woo King Hei 2025-10-29 14:23:25 HKT

I feel like that may have been the root cause, can you try to fix the function?

Wong Hei Leong 2025-10-29 16:32:43 HKT

The issue indeed came from the checkCard() function. I have modified the code so that after a wild card is played, Players can only play cards of colour that was declared or other Wild Cards. Try it out now!

```
@Override  
public boolean checkCard(Card matchCard) {  
    if (matchCard.color == this.color || matchCard.type == Type.WildDrawFour ||  
        matchCard.type == Type.Wild) {  
        return true;  
    }  
    return false;  
}
```

Woo King Hei 2025-10-29 17:13:49 HKT

It works fine now, thanks!

3.5 WildCard Timeout Operation

Product	UNO
Status	CLOSED FIXED
Component	WildCard.java
Version	1.0
Hardware	Macintosh
OS	Mac OS
Severity	Significant
Priority	High
Reporter	Woo King Hei <kingwoo8-c@my.cityu.edu.hk>
Assignee	Wong Hei Leong <hlwong397-c@my.cityu.edu.hk>

Woo King Hei 2025-10-29 22:14:59 HKT

Description

When testing the program, I played a Wild Card on my turn as normal. I was deciding which colour to declare and couldn't make a choice, so the timer ran out and passed a turn. The problem is that the next player cannot play any card. Also, the panel for selecting which color to declare did not disappear after the timer ended.

Steps to Reproduce

1. Play a wild card
2. Wait for the turn countdown timer to end

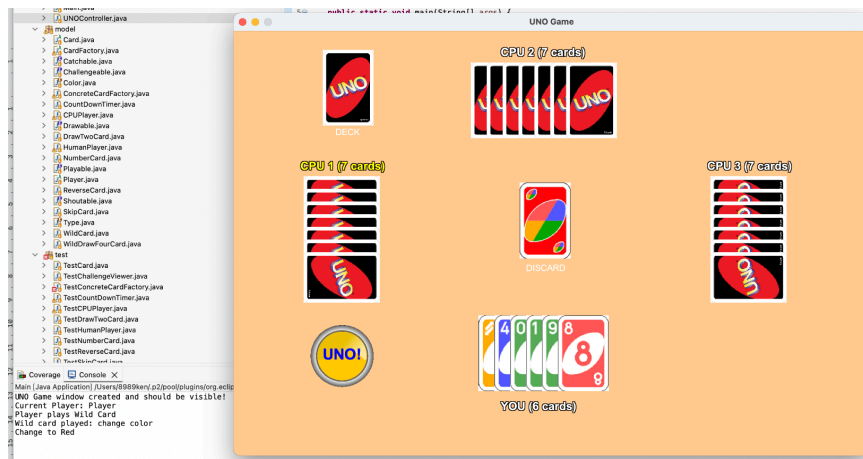
Please look into this issue and fix it as soon as possible.

Related Code

```
@Override
public void onTimerComplete() {
    if (wildCardViewer.isHavingWild()) {
        wildCardViewer.setHavingWild(hasWild: false);
        if (wildCardViewer.getCard().getType() == Type.WildDrawFour) {
            wildCardViewer.callChallenge();
        } else {
            passNextPlayer(amount: 1);
        }
    }
    eachRound(); } }
```

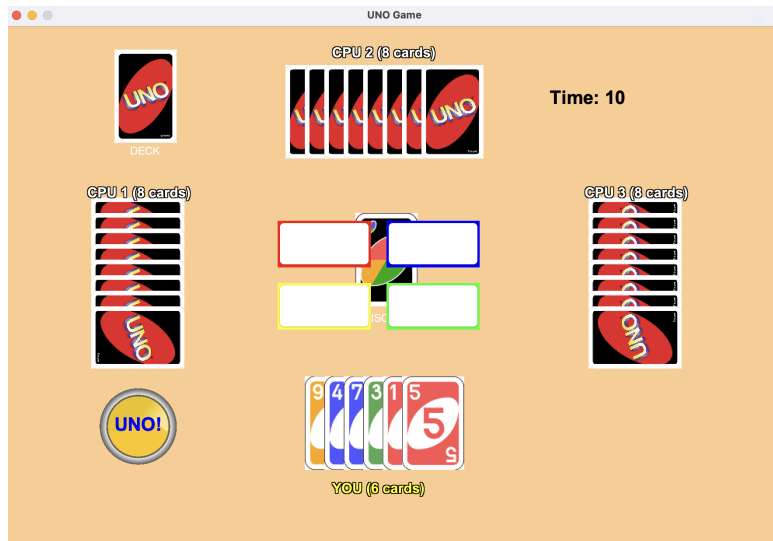
Expected Result

In this game, I played the Wild Card and stayed AFK for 10 seconds. In this situation, the default colour (red) should be selected after the timer runs out, and the appearance (colour) of the card should also be changed.



Actual Result

However, not only did the card not change its color after the timer ended, but also prevented the next player to play any cards except wild cards. The buttons are not removed after the timer ended.



Here is the game log for better understanding:

```

UNO Game window created and should be visible.
Current Player: Player
Player plays Wild Card
Wild card played: change color
CPU1 (CPU) has no valid cards to play and must draw.
CPU2 (CPU) has no valid cards to play and must draw.
CPU3 (CPU) has no valid cards to play and must draw.

```

Wong Hei Leong 2025-10-29 22:30:35 HKT

Received your bug report.

Wong Hei Leong 2025-10-29 23:35:12 HKT

After reviewing UNOController.java, I have found that the bug is due to not properly setting the timeout function. I should modify the function so it calls the function of removing the window first. Moreover, I should create a function for auto-selecting the default colour (red) when facing timeout using Wild Cards. Or else, the selected color will become null such that only wild cards are acceptable for the next player.

Wong Hei Leong 2025-10-30 22:21:29 HKT

I have fixed the issue. It will automatically select red and the wild card will change to red so that the next player can play wild card and red cards.

@Override

```
@Override
public void onTimerComplete() {
    if (wildCardViewer.isHavingWild()) {
        wildCardViewer.autoSelectColor();
        wildCardViewer.setHavingWild(hasWild: false);
        wildCardViewer.removeButtons();
        if (wildCardViewer.getCard().getType() == Type.WildDrawFour) {
            wildCardViewer.callChallenge();
        } else {
            passNextPlayer(amount: 1);
        }
    }
    eachRound();
}
```

```
public void autoSelectColor() {
    wild.setColor(model.Color.Red);
    System.out.println(x: "Change to Red");
    if(this.wild.getType()==Type.WildDrawFour){
        ((Card)wild).loadImage(path: "/asset/uno-card-images-master/Wild_Card_Draw_4_Red.jpg");
    }else{
        ((Card)wild).loadImage(path: "/asset/uno-card-images-master/Wild_Card_Change_Colour_Red.jpg");
    }
}
```

Woo King Hei 2025-10-30 23:12:25 HKT

Yep, the bug is solved! Thanks!

3.6 Cannot shout UNO when other player's turn

Product	UNO
Status	CLOSED FIXED
Component	HumanPlayer.shoutUno()
Version	1.0
Hardware	Macintosh
OS	Mac OS
Severity	Significant
Priority	High
Reporter	Woo King Hei <kingwoo8-c@my.cityu.edu.hk>
Assignee	Wong Hei Leong <hlwong397-c@my.cityu.edu.hk>

Woo King Hei 2025-11-02 15:09:27 HKT

Description

When testing the program, I had 2 cards left in hand and forgot to shout UNO on my turn. I wanted to perform the action in another player's turn, but I was unable to do it.

Steps to Reproduce

1. Play the cards until remain 2 cards (condition for shouting UNO)
2. Do not shout UNO in your turn and play a card
3. Click the shout UNO button in other players' turn

Please look into this issue and fix it as soon as possible.

Related Code

```
@Override
public String shoutUno() {
    String errorMessage = null;

    // Only current player can shout UNO
    if (this != controller.getCurrentPlayer()) {
        errorMessage = "It's not your turn!";
    } else if (this.getIsShout()) {
        errorMessage = "You shouted UNO already!";
    } else if (this.getHand().size() != 1) {
        errorMessage = "You have more than one card!";
    } else {
        isShout = true;
        System.out.println(name + " shouts UNO!");
    }

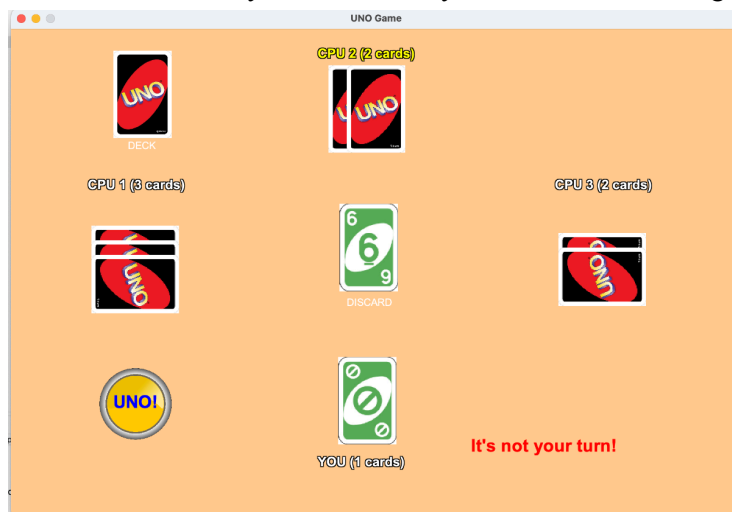
    return errorMessage;
}
```

Expected Result

In this game, my name tag should exist “*UNO*” after clicking the “UNO” button in other players’ turn when I fulfill the condition of shoutUNO.

Actual Result

However, It will only show “It’s not your turn” after clicking the button



Wong Hei Leong 2025-11-02 16:33:17 HKT

Received your bug report.

Wong Hei Leong 2025-11-02 16:57:35 HKT

I looked a bit into HumanPlayer.java where I found that the shoutUno() function has incorrect condition checking. I think the conditional `if (this != controller.getCurrentPlayer())` prevented HumanPlayer from calling UNO outside of their turn, so fixing this would resolve the issue completely. What do you think?

Woo King Hei 2025-11-02 23:12:27 HKT

Sounds great, I think changing the conditional to check whether HumanPlayer has 1 card left in hand will solve the issue.

Wong Hei Leong 2025-11-02 23:20:39 HKT

Got it. I will notify you when the bug is fixed.

Wong Hei Leong 2025-11-03 23:13:46 HKT

I have fixed the issue. HumanPlayer now can shout UNO with the correct condition.

```
@Override
public String shoutUno() {
    String errorMessage = null;
    if (this != controller.getCurrentPlayer() && this.getHand().size() > 1) {
        errorMessage = "You have more than 1 card!";
    } else if (this != controller.getCurrentPlayer() && this.getHand().size() == 1) {
        isShout = true;
        System.out.println(name + " shouts UNO!");
    } else if (this.getIsShout()){
        errorMessage = "You shouted UNO already!";
    } else if (this.getHand().size() > 2) {
        errorMessage = "You have more than 2 card!";
    } else {
        boolean hasPlayableCard = false;
        for (Card c : this.getHand()){
            if(controller.canPlayCard(c, controller.getTopCard(index: 1))){
                hasPlayableCard = true;
            }
        }
        if (!hasPlayableCard) {
            errorMessage = "You have no playable card!";
            return errorMessage;
        }
        isShout = true;
        System.out.println(name + " shouts UNO!");
    }
    return errorMessage;
}
```

Woo King Hei 2025-11-04 00:02:25 HKT

It works now, thanks!

3.7 “Null Card” is Being Added to the Hand When the Deck is Empty

Product	UNO
Status	CLOSED FIXED
Component	UNOController.java
Version	1.0
Hardware	Macintosh
OS	Mac OS
Severity	Moderate
Priority	Medium
Reporter	Woo King Hei <kingwoo8-c@my.cityu.edu.hk>
Assignee	Wong Hei Leong <hlwong397-c@my.cityu.edu.hk>

Woo King Hei 2025-11-04 15:45:13 HKT

Description

While I was testing out the UNO game, sometimes the game would go to the point where all cards had been drawn from the deck. In this situation, when a player draws a card, they will get a card which the GUI cannot render, probably because the “card” is a Null object. Though it is very unlikely for this to occur in an actual game since it is very hard to deck out.

Steps to Reproduce

1. Play the game normally to a point where there are no cards left in the deck.
2. Attempt to draw a card.

Please look into this issue and fix it as soon as possible.

Related Code

```
public void getCardFromDeck() {
    if (currentPlayer != null) {
        System.out.println(currentPlayer.getName() + " draws a card from the deck.");
        currentPlayer.drawCard(cardFactory.giveCard(Deck, isTop: false,
            checkCurrentPlayer() == 0 ? true : false, cardString: ""));
        Card card = currentPlayer.getHand().get(currentPlayer.getHand().size() - 1);
        gamePanel.updateDisplay();
        if (canPlayCard(card, getTopCard())) {
            if (checkCurrentPlayer() == 0) {
                deckPlayCardViewer.setIsDeciding(card);
                turnTimer.startTimer(seconds: 10);
            } else {
                System.out.println(currentPlayer.getName() +
                    " got a matching card! they choose to play the card");
                currentPlayer.playCard(card);
            }
        } else {
            passNextPlayer(amount: 1);
            eachRound();
        }
    } else {
        System.err.println(x: "currentPlayer variable is null in getCardFromDeck()");
    }
}
```

Expected Result

In a particularly long game, the deck becomes empty and the game should end, deciding the result by the number of cards in hand.

Actual Result

However, I believe `NullPointerException` occurred and the game is stuck.

Here is the game log for better understanding:

A screenshot of a Java IDE window titled "src — java.controller.Main — 80x24". The editor displays a series of 16 identical warning messages from the compiler. Each message reads: "Warning: Invalid card index 1. PlayedCard size is 0". These warnings are listed vertically, one per line, indicating a problem with the code being compiled. The IDE interface includes standard OS window controls (red, yellow, green buttons) at the top left.

Wong Hei Leong 2025-11-04 16:17:35 HKT

Received your bug report.

Wong Hei Leong 2025-11-04 16:58:21 HKT

After reviewing the code, this happened because the function did not check for the case when a null card is being drawn from the deck. Since this is a relatively simple bug, I already implemented the check and fixed the issue. Test it out and inform me if it works properly now or not.

Woo King Hei 2025-11-04 17:34:13 HKT

Yes, now it works properly, thanks!

3.8 The Top Card is Not a Number Card During Game Initialization

Product	UNO
Status	CLOSED FIXED
Component	UNOController.java
Version	1.0
Hardware	Macintosh
OS	MacOS
Severity	Minor
Priority	Medium
Reporter	Wong Hei Leong <hlwong397-c@my.cityu.edu.hk>
Assignee	Chan Wing Ki <wkchan597-c@my.cityu.edu.hk>

Wong Hei Leung 2025-11-09 13:23:56

Description

Sometimes when the game starts, the first card (the card that is on the table) is not a number card, the card is instead an action card such as Skip and Reverse. It's not a major bug but I'm not sure which part is responsible for this.

Steps to Reproduce

1. Start a new UNO game.
2. Observe the card on the table, it may not be a number card.

Please look into the issue and fix it as soon as possible.

Related Code

(Reporter couldn't locate)

Expected Result

When the game starts, the top card should be a number card.

Actual Result

However, for this game, the top card is a red Skip Card.



Chan Wing Ki 2025-11-09 13:30:12

Received your bug report.

Chan Wing Ki 2025-11-09 13:30:12

I have looked into the code, the problem is the `canPlayCard()` function, right now when the game starts it just accepts any card. That's why the top card can sometimes be an Action card when the game initializes.

Chan Wing Ki 2025-11-09 13:50:43

I have fixed the code, now the function implements a type check so the first card is for sure a number card.

Wong Hei Leung 2025-11-09 14:16:52

Bug fixed! Thanks!

3.9 Game Does Not End When Current Player has No Cards in Hand

Product	UNO
Status	CLOSED FIXED
Component	UNOController.java
Version	1.0
Hardware	Macintosh
OS	MacOS
Severity	Significant
Priority	High
Reporter	Woo King Hei <kingwoo8-c@my.cityu.edu.hk>
Assignee	Wong Hei Leong <hlwong397-c@my.cityu.edu.hk>

Woo King Hei 2025-11-16 14:45:59 HKT

Description

I was playing a UNO game normally up until the point where a player played his last card. The game should end immediately, however the game somehow continued and the next player played a card as usual.

Steps to Reproduce

1. Play until another player played his last card
2. Watch the game continue as normal even if there is a player with no cards in hand.

Please look into the issue and fix it as soon as possible.

Related Code

```

public void playCard(Card card) {
    if (card == null) {
        System.err.println(x: "Card variable is empty in playCard()");
        return;
    }
    isAction = true;
    card.setRotation(angle: 0);
    PlayedCard.add(card);
    currentPlayer.playCard(card);
    gamePanel.updateDisplay();
    card.cardFunction(this);
    isAction = false;
    if(currentPlayer.getHand().size() == 0) {
        System.out.println(currentPlayer.getName() + " win!");
        gamePanel.setIsGameEnd(isGameEnd: true);
        turnTimer.stopTimer();
    }
}
}

```

Expected Result

In a regular game, CPU1 already played all its cards, the game should end immediately with the result screen.

Actual Result

However, the game did not end and continued, as shown in the screenshot below:



Wong Hei Leong 2025-11-16 15:12:26 HKT

Received your bug report.

Wong Hei Leong 2025-11-16 15:46:13 HKT

After reviewing the playCard() function, I believe the issue occurred because the card playing action is sequenced before the isGameEnd check, therefore when the check operates, it is already the next player's turn so it checks if the next player has no cards in hand instead.

Wong Hei Leong 2025-11-16 17:35:56 HKT

I have fixed the issue, the function now checks if the game has ended before updating the game state and passing to the next player, try it out!

Woo King Hei 2025-11-16 18:02:28 HKT

The game now works fine, thanks!

3.10 HumanPlayer Can Draw Cards Multiple Times During Their Turn

Product	UNO
Status	CLOSED FIXED
Component	UNOController.java
Version	1.0
Hardware	Macintosh
OS	MacOS
Severity	Moderate
Priority	Medium
Reporter	Woo King Hei <kingwoo8-c@my.cityu.edu.hk>
Assignee	Wong Hei Leong <hlwong397-c@my.cityu.edu.hk>

Woo King Hei 2025-11-23 17:57:23 HKT

Description

During a game I tested the drawing card functionality by clicking the deck multiple times during my turn. Turns out that if the card I drew is a playable card, I can draw multiple cards if I click the deck quick enough. I think the issue originates from the draw card function.

Steps to Reproduce

1. Wait for your turn where you have no playable cards
2. Draw a card
3. If the card is playable, click the deck rapidly to draw multiple cards

Please look into the issue and fix it as soon as possible.

Related Code

```
public void getCardFromDeck() {
    if (currentPlayer != null) {
        System.out.println(currentPlayer.getName() + " draws a card from the deck.");
        currentPlayer.drawCard(cardFactory.giveCard(Deck, isTop: false,
            checkCurrentPlayer() == 0 ? true : false, cardString: ""));
        Card card = currentPlayer.getHand().get(currentPlayer.getHand().size() - 1);
        gamePanel.updateDisplay();
        if (canPlayCard(card, getTopCard())) {
            if (checkCurrentPlayer() == 0) {
                deckPlayCardViewer.setIsDeciding(card);
                turnTimer.startTimer(seconds: 10);
            } else {
                System.out.println(currentPlayer.getName() +
                    " got a matching card! they choose to play the card");
                currentPlayer.playCard(card);
            }
        } else {
            passNextPlayer(amount: 1);
            eachRound();
        }
    } else {
        System.err.println(x: "currentPlayer variable is null in getCardFromDeck()");
    }
}
```

Expected Result

In this game, it's my turn and I have 10 cards in hand. I chose to draw a card. In this case, regardless of the number of times I click the deck, I should only draw one card.



Actual Result

However, I drew multiple cards as I clicked the deck multiple times, as shown below:



Wong Hei Leong 2025-11-23 18:30:12 HKT

Received your bug report.

Wong Hei Leong 2025-11-23 19:03:36 HKT

Thanks to you locating the `getCardFromDeck()` function, I have quickly fixed the issue. The function now has a `isAction` boolean to lock the draw card action. With this implementation, the player can only draw cards if they have not done the action for their turn yet (e.g playing a card, drawing a card)

```

public void getCardFromDeck() {
    if (!isAction) {
        isAction = true;
        if (currentPlayer != null) {
            System.out.println(currentPlayer.getName() + " draws a card from the deck.");
            Card newCard = cardFactory.giveCard(Deck, isTop: false,
                checkCurrentPlayer() == 0 ? true : false, cardString: "");
            if (newCard == null) {
                deckEmpty();
                isAction = false;
                return;
            }
            currentPlayer.drawCard(newCard);
            gamePanel.updateDisplay();
            if (canPlayCard(newCard, getTopCard(index: 1))) {
                if (checkCurrentPlayer() == 0) {
                    deckPlayCardViewer.setIsDeciding(newCard);
                    turnTimer.startTimer(seconds: 10);
                } else {
                    System.out.println(
                        currentPlayer.getName() +
                        " got a matching card! they choose to play the card");
                    currentPlayer.playCard(newCard);
                }
            } else {
                passNextPlayer(amount: 1);
                eachRound();
                isAction = false;
            }
        }
    }
}

```

Woo King Hei 2025-11-23 20:41:54 HKT

I tried again and now the draw card function works normally, thanks!

4. Appendix

4.1 Bug Report Template

<Bug Name>

Product	UNO
Status	
Component	
Version	
Hardware	
OS	
Severity	
Priority	
Reporter	
Assignee	

<Reporter Name> <Time>

<Description>

Describe the situation caused by the bug.

<Related Code>

The reporter can paste related code components here (if possible) to help the assignee to find the bug faster.

<Expected Result>

The supposed outcome in the situation.

<Actual Result>

The actual outcome in the situation due to the bug.